

Universidad de Sevilla
Máster en Ingeniería de Electrónica, Robótica y Automática (MIERA)

Percepción en Automática y Robótica

Simulación de Coche Autónomo: Detección de Carriles y Señales

Grupo XI

Samuel Boleslaw Locoche
Víctor Javier Granero Gil
José Francisco López Ruiz

Curso 2025–2026

Índice

1. Introducción	3
2. Detección de carriles	4
2.1. Primer enfoque: estimación mediante líneas horizontales	4
2.2. Enfoque basado en segmentación semántica	5
2.2.1. Encoder VGG16	5
2.2.2. Decoder U-Net	5
2.2.3. Bloque convolucional	6
2.3. Entrenamiento del modelo	6
2.4. Resultados del seguimiento de carriles	7
3. Detección y clasificación de señales	8
3.1. Detección de las señales	8
3.2. Clasificación de las señales	10
3.2.1. Datasets utilizados	10
3.2.2. Arquitectura de la red neuronal	11
3.2.3. Resultados de los entrenamientos	12
3.2.4. Prueba sobre señales de la simulación	13
4. Integración en simulación y resultados	13
4.1. Entorno de simulación: CARLA Simulator	13
4.2. Arquitectura del sistema	14
4.3. Infraestructura: Docker y contenedores	15
4.4. Nodos ROS2 del sistema	16
4.4.1. Nodo de detección de carriles (<code>lane_detection_node</code>)	16
4.4.2. Nodo de detección de señales (<code>sign_detection_node</code>)	16
4.4.3. Nodo de control del vehículo (<code>vehicle_control_node</code>)	16
4.4.4. Nodo generador de anotaciones (<code>annotation_generator_node</code>)	17
4.5. Generación de datasets sintéticos	18
4.6. Resultados	19
4.6.1. Rendimiento en tiempo real	19
4.6.2. Entorno de pruebas	19
4.6.3. Conducción autónoma en simulación	19
5. Conclusiones	19

Resumen

Este trabajo presenta el diseño e implementación de un sistema de percepción para un vehículo autónomo simulado en *CARLA Simulator*, integrado con *ROS2 Humble* mediante contenedores Docker. Tras evaluar un primer enfoque basado en líneas horizontales, la detección de carriles se resuelve mediante una red VGG16–U-Net entrenada con el *dataset* TuSimple (TensorFlow/Keras) y exportada a ONNX para inferencia en tiempo real. La detección y clasificación de señales de límite de velocidad (30, 60 y 90 km/h) combina localización por visión clásica (OpenCV en el desarrollo del clasificador; segmentación semántica de CARLA en simulación) con una CNN entrenada en PyTorch sobre GTSRB y un conjunto propio de imágenes. Ambos módulos se integran en un sistema ROS2 con controladores PID de velocidad y dirección, con latencias medias de unos 50 ms en carriles y 13 ms en señales. El código fuente, modelos entrenados, *datasets* y vídeos demostrativos se encuentran en el repositorio público de GitHub: <https://github.com/JoseLopez36/Navegacion-Deteccion-Senales>.

1. Introducción

La conducción autónoma es uno de los retos tecnológicos más ambiciosos de la última década. Para que un vehículo sea capaz de desplazarse sin intervención humana debe ser competente en varias etapas: percepción del entorno, planificación de trayectoria y actuación sobre los sistemas de control. Este trabajo aborda la etapa de *percepción*, responsable de extraer información útil del entorno a partir de los datos sensoriales.

El objetivo del proyecto es desarrollar un sistema de percepción y control básico para un vehículo autónomo simulado en *CARLA Simulator*, conectado a través de *ROS2 Humble*. El flujo de funcionamiento diseñado es el siguiente:

1. Adquisición de imágenes desde la cámara virtual de CARLA.
2. Detección de líneas de carril mediante segmentación semántica (VGG16–U-Net entrenada con el *dataset* TuSimple en TensorFlow/Keras, inferida en ONNX) y estimación del error lateral del vehículo.
3. Localización de señales de tráfico en la imagen y clasificación de los límites de velocidad (30, 60 y 90 km/h) mediante una CNN entrenada en PyTorch con GTSRB y un *dataset* propio.
4. Adaptación de la velocidad de referencia en función de las señales detectadas.
5. Cálculo de los comandos de dirección y velocidad mediante controladores PID y publicación hacia CARLA.

Todo el código fuente, modelos entrenados, *datasets* y material multimedia (imágenes y vídeo demostrativo) están disponibles en el repositorio público de GitHub del proyecto:

<https://github.com/JoseLopez36/Navegacion-Deteccion-Senales>

Estructura de la memoria

La memoria se organiza en tres bloques principales:

Sección 2: Detección de carriles. Presenta el *dataset* TuSimple, un primer enfoque descartado basado en líneas horizontales, la arquitectura VGG16–U-Net adoptada, el proceso de entrenamiento y los resultados del modelo de segmentación en distintos escenarios.

Sección 3: Detección y clasificación de señales. Describe la detección de señales mediante OpenCV, los *datasets* empleados (GTSRB y conjunto propio para 90 km/h), la arquitectura y entrenamiento de la CNN clasificadora, y los resultados obtenidos en el conjunto de prueba capturado en la simulación.

Sección 4: Integración en simulación y resultados. Desarrolla la arquitectura software completa en CARLA/ROS2/Docker, los nodos del sistema, los controladores PID de velocidad y dirección, la generación de *datasets* sintéticos y los resultados de conducción autónoma en tiempo real.

2. Detección de carriles

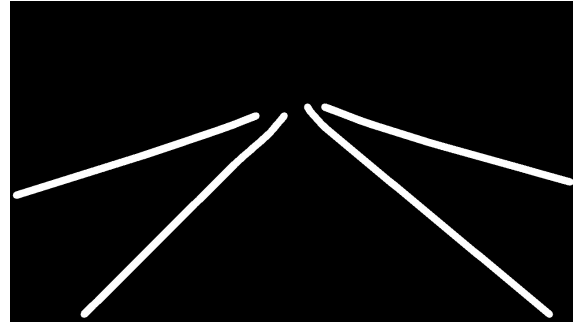
Uno de los objetivos principales de este proyecto es la detección de las líneas de carril a partir de imágenes capturadas por una cámara frontal. Para ello se han evaluado diferentes enfoques basados en visión artificial y aprendizaje profundo, con el objetivo de obtener una representación fiable de la posición de los carriles que posteriormente pueda utilizarse para tareas de guiado y seguimiento.

Para el entrenamiento y validación de los modelos se ha utilizado el *dataset* TuSimple, uno de los conjuntos de datos de referencia en el ámbito de la detección de carriles para conducción autónoma.

Este conjunto proporciona imágenes capturadas desde una cámara frontal instalada en un vehículo, así como la posición anotada de las líneas de carril presentes en cada escena.



(a) Imagen de ejemplo del *dataset*.



(b) Máscara generada a partir de JSON.

Figura 1: Ejemplo de una imagen del conjunto de datos y la máscara de segmentación obtenida a partir de sus anotaciones en formato JSON.

2.1. Primer enfoque: estimación mediante líneas horizontales

Como primera aproximación se intentó reproducir la metodología propuesta por Yoo et al.¹, basada en la representación de los carriles mediante una serie de puntos obtenidos a partir de múltiples líneas horizontales distribuidas sobre la imagen.

La idea principal consiste en dividir la imagen en diferentes franjas horizontales y estimar para cada una de ellas la posición horizontal de las líneas de carril. De este modo, el carril queda representado como una sucesión de puntos que posteriormente pueden unirse para reconstruir la geometría de los carriles.

Debido a las limitaciones computacionales y al tamaño del modelo utilizado, inicialmente se emplearon únicamente entre ocho y diez líneas horizontales distribuidas a lo largo de la imagen.

Sin embargo, esta resolución resultó insuficiente para describir adecuadamente la geometría de los carriles. En situaciones con curvas pronunciadas o cambios de perspectiva,

¹Yoo, S., Lee, H., Myeong, H., Yun, S., Park, H., Cho, J., & Kim, D. H. (2020). *End-to-End Lane Marker Detection via Row-wise Classification*. arXiv:2005.08630.

la cantidad de puntos obtenida era demasiado reducida para reconstruir de forma precisa la trayectoria de las líneas.

Tras diversas pruebas se comprobó que los resultados obtenidos no alcanzaban el nivel de precisión requerido para el seguimiento fiable del vehículo, por lo que se decidió explorar un enfoque alternativo basado en segmentación semántica.

2.2. Enfoque basado en segmentación semántica

La segunda aproximación consistió en reformular el problema como una tarea de segmentación semántica binaria.

En lugar de estimar únicamente una serie de puntos representativos, el objetivo pasa a ser clasificar cada píxel de la imagen indicando si pertenece o no a una línea de carril.

Esta estrategia permite obtener una representación mucho más detallada de la escena y facilita la posterior extracción de la geometría del carril.

Para este propósito se desarrolló una arquitectura basada en U-Net utilizando VGG16 como encoder.

La arquitectura implementada combina dos componentes principales:

- Encoder basado en VGG16.
- Decoder basado en U-Net.

El encoder se encarga de extraer características visuales de la imagen, mientras que el decoder reconstruye progresivamente una máscara de segmentación con la misma resolución que la imagen de entrada.

2.2.1. Encoder VGG16

Se ha utilizado la arquitectura VGG16 preentrenada sobre ImageNet mediante técnicas de *transfer learning*.

El uso de pesos preentrenados permite aprovechar características visuales previamente aprendidas, tales como bordes, texturas y patrones geométricos, reduciendo el tiempo de entrenamiento y mejorando la capacidad de generalización.

Durante el proceso de entrenamiento se permitió el ajuste de todos los parámetros de la red para adaptarlos específicamente a la detección de carriles.

Las capas seleccionadas del encoder se utilizan posteriormente en las conexiones de salto (*skip connections*) características de la arquitectura U-Net.

2.2.2. Decoder U-Net

El decoder reconstruye progresivamente la resolución espacial perdida durante el proceso de compresión realizado por el encoder.

Para ello se utilizan operaciones de upsampling que duplican la resolución de los mapas de características en cada etapa.

Posteriormente, dichos mapas se concatenan con la información procedente de las capas equivalentes del encoder mediante *skip connections*.

Estas conexiones permiten recuperar detalles espaciales finos que resultan fundamentales para delimitar con precisión las líneas de carril.

2.2.3. Bloque convolucional

Tras cada operación de concatenación se aplica un bloque convolucional compuesto por:

- Convolución 1×1 .
- Convolución 3×3 .
- Activación ELU.
- Batch Normalization.
- Dropout.

La convolución 1×1 permite reorganizar y combinar la información procedente de los distintos canales de características.

Posteriormente, la convolución 3×3 analiza las relaciones espaciales locales y permite detectar patrones relevantes para la identificación de los carriles.

La activación ELU introduce no linealidad en el modelo y mejora la estabilidad del aprendizaje frente a funciones de activación tradicionales como ReLU.

La capa Batch Normalization normaliza las activaciones de la red, acelerando la convergencia durante el entrenamiento y mejorando la robustez del modelo.

Finalmente, se incorpora una capa Dropout con una tasa del 25 %, utilizada como mecanismo de regularización para reducir el sobreajuste.

2.3. Entrenamiento del modelo

Para entrenar el modelo fue necesario transformar las anotaciones proporcionadas por TuSimple en máscaras binarias de segmentación.

Cada máscara contiene información píxel a píxel indicando las regiones pertenecientes a las líneas de carril. Estas máscaras constituyen la salida esperada durante el proceso de entrenamiento supervisado.

El entrenamiento se realizó utilizando imágenes redimensionadas a una resolución de 224×224 píxeles.

Durante el proceso de aprendizaje, la red recibe como entrada una imagen de carretera y genera una máscara de probabilidad donde cada píxel representa la probabilidad de pertenecer a una línea de carril.

La salida final se obtiene aplicando una función de activación sigmoide y un umbral de decisión para convertir dichas probabilidades en una máscara binaria.

El entrenamiento se realizó durante 10 épocas. Las curvas de precisión y pérdida se representan en la Figura 2, donde se aprecia cómo mejora el rendimiento del modelo a medida que avanzan las épocas.

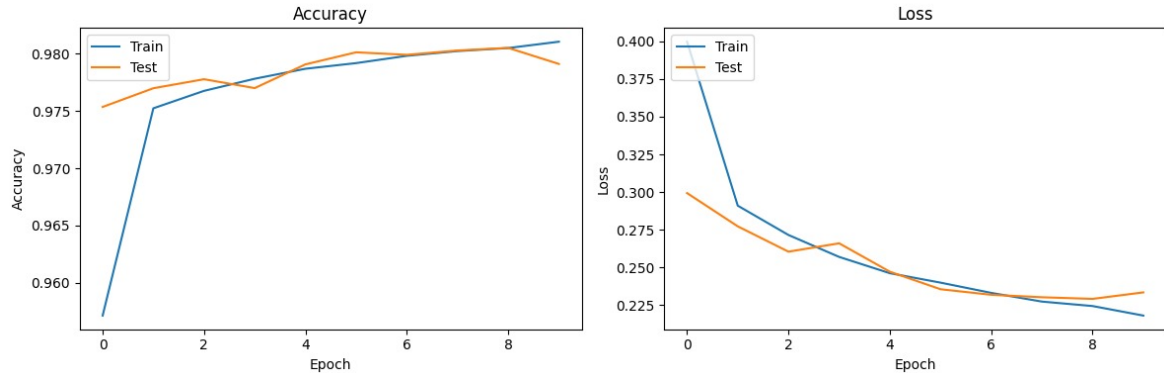


Figura 2: Evolución de la precisión y la pérdida durante el entrenamiento.

2.4. Resultados del seguimiento de carriles

En esta sección se presentan los resultados obtenidos por el modelo de detección de carriles en distintos escenarios de prueba.

En primer lugar, se muestra una predicción sobre una carretera recta, donde el modelo es capaz de identificar de forma estable la estructura del carril sin presentar desviaciones significativas.

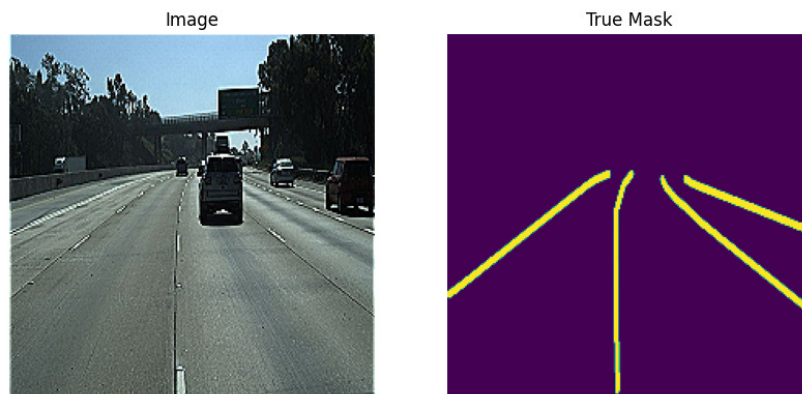


Figura 3: Predicción sobre un carril recto.

En segundo lugar, se analiza una predicción en un tramo con una ligera curva. En este escenario se observa que el modelo mantiene un buen seguimiento de la trayectoria del carril, adaptándose correctamente a la curvatura sin perder continuidad, lo que indica una adecuada capacidad de generalización ante variaciones suaves en la geometría de la carretera.

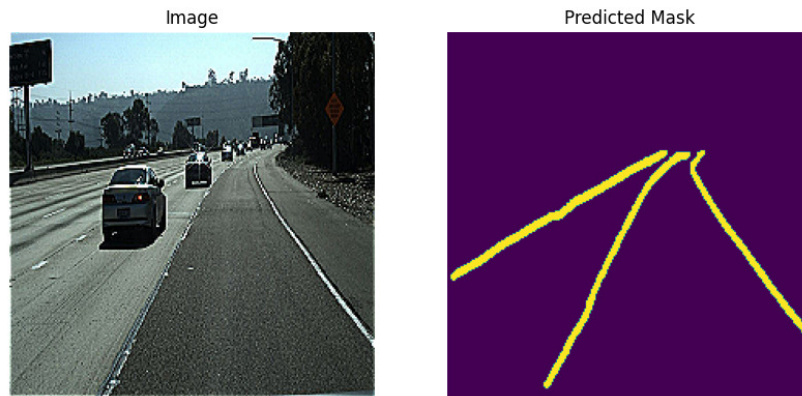


Figura 4: Predicción sobre un carril con ligera curva.

Por último, se incluye una comparación entre la predicción del modelo y la máscara real correspondiente. En este caso se aprecia una alta similitud entre ambas, con una buena coincidencia en la forma y posición de los carriles. No obstante, se observan pequeñas desviaciones en algunas zonas, debidas a elementos presentes en la imagen.

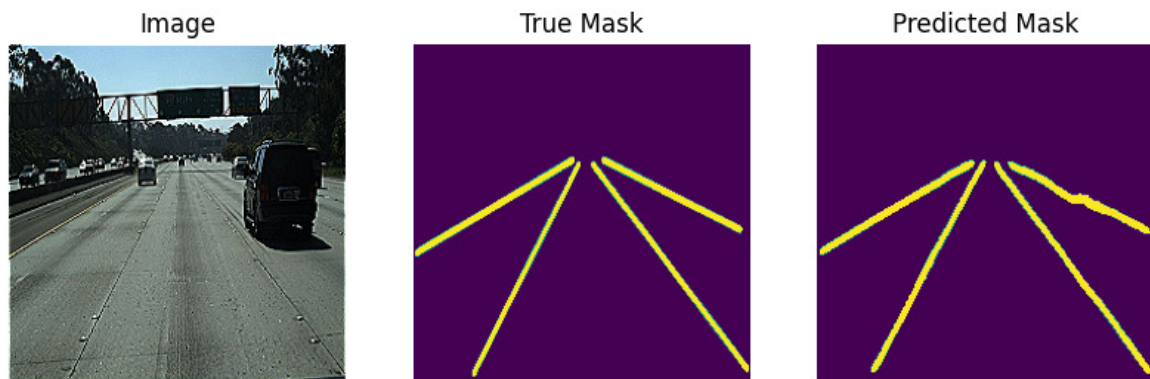


Figura 5: Comparación entre imagen de entrada, máscara real y predicción del modelo.

3. Detección y clasificación de señales

Una vez que el vehículo puede seguir el carril, debe respetar las reglas del código de circulación para evitar accidentes. Por ello, tiene que ser capaz de detectar y reconocer las señales. En esta parte se tratan tres señales de límite de velocidad: 30, 60 y 90 km/h. En las subsecciones siguientes se explica cómo se localizan las señales en la imagen y cómo se clasifican mediante una red neuronal convolucional.

3.1. Detección de las señales

En cuanto a la detección de las señales, se utiliza el módulo *OpenCV* de Python para procesar las imágenes. En esta sección se describen los distintos pasos del algoritmo de localización empleado durante el desarrollo del clasificador. La Figura 6 ilustra una imagen que podría proceder de la cámara del vehículo y que el algoritmo recibe como entrada.



Figura 6: Imagen de entrada de la detección, sobre la que se realizarán los distintos pasos del algoritmo.

En primer lugar, se detectan todas las zonas rojas de la imagen. En efecto, las tres señales comparten el rojo como color distintivo. Se convierte la imagen al formato HSV (más robusto frente a los cambios de iluminación) y se obtiene una máscara del color rojo. La Figura 7 muestra la máscara obtenida tras este paso.



Figura 7: Máscara obtenida mediante la detección del color rojo.

Una vez detectadas las zonas rojas, se calculan sus contornos. El objetivo es conservar la zona de mayor área, pues si hay una señal en la imagen, corresponde a esta región. No obstante, se define un área mínima de detección: si la zona mayor tiene un área inferior a ese umbral, se considera que la imagen no contiene ninguna señal.

Si se detecta una señal, se crea una *bounding box* alrededor de ella y se amplía el recorte correspondiente. Este recorte constituye el resultado de la detección, que permitirá a la red neuronal clasificar la señal detectada.

(a) *Bounding box* alrededor de la señal.

(b) Zoom sobre la señal.

Figura 8: Resultados de la detección.

En el sistema integrado (Sección 4), la localización de señales en tiempo real se realiza mediante la segmentación semántica de CARLA, más fiable que el pipeline basado en color en el entorno simulado. El clasificador CNN descrito a continuación se aplica en ambos casos sobre el recorte obtenido.

3.2. Clasificación de las señales

Ahora que el vehículo es capaz de detectar la presencia o la ausencia de una señal, debe reconocerla para adaptar su velocidad a la situación. En esta parte se presentan la arquitectura CNN del modelo, así como los *datasets* utilizados para entrenarlo y validarlo. También se analizan los resultados para identificar las ventajas y los inconvenientes del modelo.

3.2.1. Datasets utilizados

Se utiliza el *German Traffic Sign Recognition Benchmark* (GTSRB): <http://benchmark.ini.rub.de/?section=gtsdb&subsection=news>

Este *dataset* contiene varios tipos de señales, como señales de velocidad o de dirección. Sin embargo, no incluye señales de 90 km/h, por lo que se utiliza únicamente para las clases de 30 y 60 km/h.

Para crear un *dataset* de entrenamiento con señales de 90 km/h, se recopilaron en primer lugar unas 70 imágenes obtenidas de Internet. A continuación, se empleó un script en Python para aplicar transformaciones a las imágenes, como rotaciones o cambios de iluminación, hasta obtener 500 imágenes en total.

Se podrían generar más imágenes mediante estas transformaciones para alcanzar 1500 y equilibrar el *dataset*. No obstante, ello provocaría un sobreajuste (*overfitting*) en la clase de las señales de 90 km/h: aunque hubiera más imágenes, todas procederían de un conjunto reducido de 70 originales. El modelo acabaría memorizando esas imágenes en lugar de generalizar. Por ello, se optó por generar menos imágenes para obtener mejores resultados.

En cuanto a los *datasets* de validación, se aplicó el mismo método para obtener imágenes de 90 km/h. En este caso, el problema anterior no resulta relevante, ya que el modelo

no ajusta sus pesos con estos datos. Así se obtuvo un *dataset* de validación con 720 imágenes para cada tipo de señal.

Por último, el *dataset* de prueba se compone únicamente de imágenes procedentes de la simulación, con 22 imágenes para cada tipo de señal.

La Tabla 1 resume el contenido de cada conjunto de datos.

Señal	Entrenamiento	Validación	Prueba
	1500	720	22
	1360	720	22
	500	720	22

Cuadro 1: Número de imágenes por señal y partición del *dataset*.

3.2.2. Arquitectura de la red neuronal

Se ha creado una red neuronal convolucional mediante el módulo *PyTorch*. El modelo debe reconocer la señal en una imagen en color de tamaño 96×96 . Como en cualquier CNN, puede dividirse en dos partes:

- La parte convolucional, compuesta por cuatro capas convolucionales con un *kernel* de 3×3 y, entre cada capa, una capa de *MaxPooling* de tamaño 2 para reducir el número de datos y, por tanto, el coste computacional.
- La parte de clasificación, compuesta por tres capas densas. Las dos primeras incluyen una capa de *Dropout* para reducir el sobreajuste, y la capa de salida dispone de 3 neuronas, una para cada clase.

Entre ambas partes, una capa **Flatten** aplanar los datos para obtener una entrada válida para las capas densas.

La Figura 9 ilustra la arquitectura del modelo.

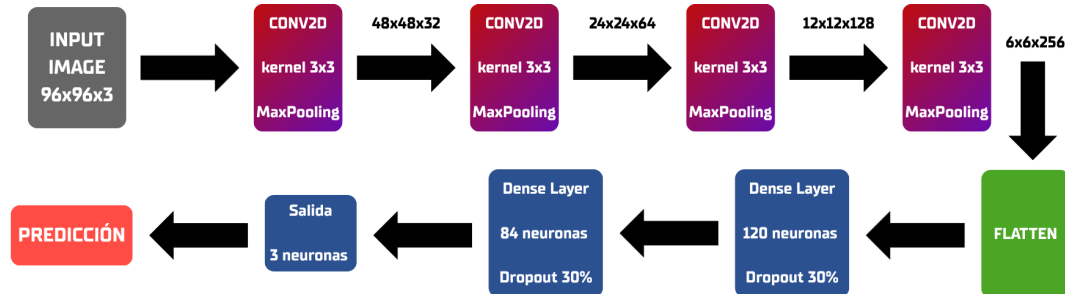


Figura 9: Arquitectura del modelo CNN empleado para clasificar las señales.

3.2.3. Resultados de los entrenamientos

En la Figura 10 se muestran los resultados de un entrenamiento del modelo con 23 épocas. Se observa que las pérdidas y la precisión se estabilizan al final del entrenamiento. Sin embargo, durante el proceso, ambas magnitudes varían de manera significativa. Estas variaciones pueden explicarse por las capas de *Dropout*: en nuestro modelo, desactivan neuronas con una probabilidad de 0.3 para combatir el sobreajuste. Esto obliga al modelo a reconocer las características de la imagen de distintas maneras.

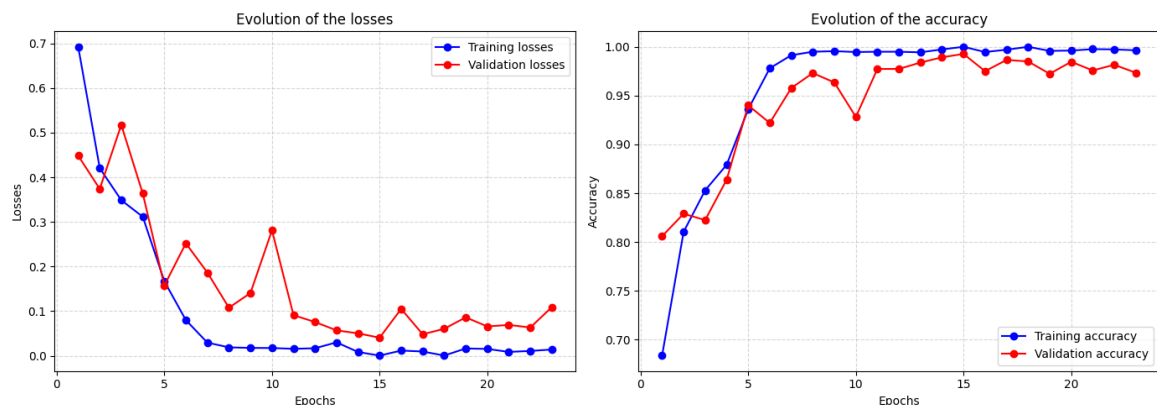


Figura 10: Evolución de las pérdidas (izquierda) y la precisión (derecha) durante un entrenamiento de 23 épocas.

Se realizaron otros entrenamientos con distintos números de épocas. Si este valor supera 25, aparece sobreajuste y los resultados sobre la clase de señales de 60 km/h empeoran notablemente. Por el contrario, con menos épocas, las pérdidas y la precisión fluctúan demasiado, como se observa en la Figura 10.

3.2.4. Prueba sobre señales de la simulación

Antes de probar el modelo en la simulación, se evaluó con un *dataset* de prueba formado por señales capturadas en el simulador. Este conjunto contiene 22 imágenes para cada tipo de señal. La Figura 11 muestra los resultados en forma de matriz de confusión.

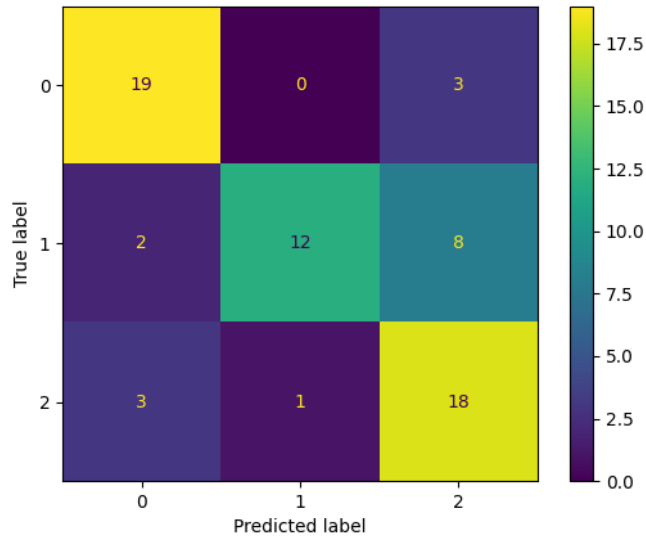


Figura 11: Matriz de confusión del conjunto de prueba

En esta matriz de confusión se observa que el modelo clasifica correctamente las señales de 30 y 90 km/h, con porcentajes de acierto del 86 % y del 81 %, respectivamente. Estos valores son aceptables, ya que en la simulación no se procesan imágenes aisladas, sino un flujo de vídeo; por ello se aplica un filtro para suavizar las predicciones.

En cuanto a las señales de 60 km/h, los resultados son más bajos, con un 54 % de aciertos. El modelo tiende a confundirlas con las de 90 km/h, probablemente porque los dígitos 6 y 9 son visualmente muy similares, lo que dificulta su clasificación.

4. Integración en simulación y resultados

Esta sección describe cómo los dos módulos de percepción anteriores se integran dentro de un sistema completo que funciona en simulación. Se presentan el entorno de simulación, la arquitectura software basada en ROS2, los nodos que componen el sistema, el controlador del vehículo y los resultados obtenidos.

4.1. Entorno de simulación: CARLA Simulator

Para el desarrollo y validación del sistema se ha empleado *CARLA Simulator* (*Car Learning to Act*), un simulador de conducción autónoma de código abierto basado en el motor gráfico *Unreal Engine 4*. CARLA proporciona sensores virtuales de gran fidelidad que permiten probar algoritmos de percepción y control sin necesidad de un vehículo real.

El simulador se ejecuta en su versión *0.9.15* dentro de un contenedor Docker con acceso a la GPU de la máquina anfitriona. Las características más relevantes del entorno configurado son:

- Mapa: Town04, una autopista con señales de tráfico y carreteras con múltiples carriles.
- Vehículo (*ego-vehicle*): modelo basado en Tesla Model 3, controlado desde ROS2.
- Sensores disponibles:
 - Cámara RGB frontal (**rgb_front**): utilizada para detección de carriles y señales.
 - Cámara de segmentación semántica (**semantic_segmentation_front**): empleada para la localización de señales en tiempo real y para la generación automática de *datasets* con *ground truth* perfecto.
 - Medidor de velocidad (**speedometer**): velocidad actual del vehículo en m/s.
 - Cámara exterior (**rgb_view**): vista en tercera persona para supervisión visual.

La Figura 12 muestra las dos perspectivas principales disponibles en el simulador.



(a) Vista exterior (tercera persona).



(b) Cámara frontal del vehículo.

Figura 12: Perspectivas en CARLA Simulator. Se aprecia el ego-vehicle tipo Tesla y, en la vista frontal, una señal de límite de velocidad a 90 km/h.

4.2. Arquitectura del sistema

El sistema completo se articula en tres capas de software que se comunican entre sí a través de una red Docker en modo *host*:

1. CARLA Simulator: simulador en contenedor Docker independiente con acceso a la GPU.
2. CARLA-ROS2 Bridge: puente oficial que traduce los datos de los sensores de CARLA en tópicos ROS2 estándar y convierte los comandos de control ROS2 en actuaciones sobre el vehículo simulado.

3. Sistema de navegación y percepción: paquete ROS2 propio (`navegacion_deteccion_senales`) que aloja todos los nodos de detección, clasificación y control.

El flujo de información entre los nodos es el siguiente: CARLA publica imágenes en `/carla/ego_vehicle/rgb_front/image` y la velocidad en `/carla/ego_vehicle/speedometer`. El `lane_detection_node` procesa esas imágenes y publica el error lateral en `/lane_detection/lane_error`. El `sign_detection_node` publica el límite de velocidad detectado en `/sign_detection/speed_limit`. El `vehicle_control_node` consume ambos tópicos para calcular y publicar los comandos de control en `/carla/ego_vehicle/vehicle_control_cmd`. Por último, el `annotation_generator_node` agrega toda la información disponible para generar una capa de visualización en tiempo real sobre *Foxglove Studio*.

4.3. Infraestructura: Docker y contenedores

Todo el sistema se orquesta mediante **Docker Compose**, levantando tres servicios con un único comando desde la raíz del repositorio:

```
1 xhost +local:docker
2 docker compose -f tools/docker-compose.yaml up
```

La siguiente tabla describe cada uno de los servicios del stack:

Cuadro 2: Servicios del *stack* Docker Compose.

Servicio	Contenedor	Función
<code>ros_bridge</code>	<code>carla_ros_bridge</code>	Puente CARLA→ROS2
<code>navegacion</code>	<code>navegacion_deteccion_senales</code>	Lanzamiento del paquete de ROS2
<code>foxglove</code>	<code>foxglove_bridge</code>	Puente WebSocket para Foxglove Studio

La imagen Docker base es `nvidia/cuda:12.8.0-cudnn-runtime-ubuntu22.04`, sobre la que se instalan:

- ROS2 Humble Hawksbill (distribución LTS sobre Ubuntu 22.04).
- PyTorch con soporte CUDA 12.8 para inferencia del clasificador de señales en GPU.
- ONNX Runtime con soporte GPU, para el modelo de detección de carriles.
- Cliente Python de CARLA 0.9.15 y `carla-ros-bridge` compilado desde el código fuente.

El código fuente del paquete ROS2 se monta como volumen dentro del contenedor de navegación, lo que permite que cualquier cambio en el host se aplique de forma inmediata (junto con `colcon build --symlink-install`) sin necesidad de reconstruir la imagen.

4.4. Nodos ROS2 del sistema

El paquete `navegacion_deteccion_senales` implementa seis nodos ROS2 en Python. Cuatro de ellos —detección de carriles, detección de señales, control del vehículo y generación de anotaciones— se lanzan mediante el fichero `run.launch.py`; los dos restantes sirven para la recolección de *datasets* sintéticos.

4.4.1. Nodo de detección de carriles (`lane_detection_node`)

Este nodo recibe cada *frame* de la cámara frontal, ejecuta la inferencia del modelo VGG16-U-Net exportado en formato ONNX y calcula el error lateral del vehículo respecto al centro del carril, expresado en metros. El error se publica en `/lane_detection/lane_error` (Float32) y el estado completo del carril —incluyendo las coordenadas de las líneas izquierda y derecha— se serializa como JSON en `/lane_detection/lane_state`.

4.4.2. Nodo de detección de señales (`sign_detection_node`)

El nodo de señales emplea dos entradas: la imagen RGB de la cámara frontal y la imagen de segmentación semántica del mismo punto de vista. La segmentación semántica identifica con precisión los píxeles correspondientes a señales de tráfico (clase de color amarillo en CARLA), obteniendo *bounding boxes* candidatas que se validan por tamaño (`min_sign_area: 150 px`). Sobre el recorte resultante se ejecuta el clasificador CNN, que determina la etiqueta de la señal (`speed_limit_30`, `speed_limit_60`, `speed_limit_90`) y el límite de velocidad en m/s.

4.4.3. Nodo de control del vehículo (`vehicle_control_node`)

Este nodo cierra el lazo de control: recibe las percepciones de los nodos anteriores y genera los comandos de actuación sobre el vehículo. Opera a 10 Hz e implementa dos controladores PID independientes.

PID de velocidad (control de crucero). Regula la velocidad del vehículo en torno a una referencia configurable, inicializada en 2.78 m/s (≈ 10 km/h). Cuando el nodo de señales detecta un límite de velocidad inferior, la referencia se reduce automáticamente. El error de control es la diferencia entre la velocidad objetivo y la velocidad real medida por el odómetro. La salida del PID se mapea a los canales *throttle* y *brake* normalizados en $[0, 1]$:

$$e_v(t) = v_{\text{ref}} - v_{\text{actual}}$$

$$u_v(t) = K_{p,v} e_v + K_{i,v} \int e_v dt + K_{d,v} \dot{e}_v$$

$$\text{throttle} = \max(0, \min(1, u_v)), \quad \text{brake} = \max(0, \min(1, -u_v))$$

PID de dirección (mantenimiento de carril). Ajusta el ángulo del volante para mantener el vehículo centrado en el carril. La entrada es el error lateral en metros.

Para suavizar la respuesta ante el ruido de la máscara de segmentación, se aplica un filtro paso-bajo de primer orden con coeficiente $\alpha = 0,05$ antes de alimentar al PID:

$$e_{\text{filt}}[k] = \alpha e_{\text{raw}}[k] + (1 - \alpha) e_{\text{filt}}[k - 1] \quad (1)$$

La salida del PID, en radianes, se normaliza por el ángulo físico máximo del volante ($\theta_{\text{máx}} = 0,6 \text{ rad} \approx 34 \text{ grados}$) para obtener el valor de dirección normalizado de CARLA en $[-1, 1]$. La convención de signo establece que un error positivo (vehículo desplazado a la derecha del carril) genera un giro a la izquierda (dirección negativa).

Ambos PIDs incorporan anti-windup que limita el término integral al rango $[-10, 10]$. Los parámetros finales configurados en `params.yaml` se recogen en la Tabla 3.

Cuadro 3: Parámetros de los controladores PID (`params.yaml`).

Parámetro	Valor	Descripción
<code>control_rate</code>	10 Hz	Frecuencia del bucle de control
<code>target_speed</code>	2.78 m/s	Velocidad objetivo ($\approx 10 \text{ km/h}$)
<code>kp_throttle</code>	0.30	Ganancia proporcional de velocidad
<code>ki_throttle</code>	0.05	Ganancia integral de velocidad
<code>kp_steering</code>	0.20 rad/m	Ganancia proporcional de dirección
<code>ki_steering</code>	0.02 rad/(m·s)	Ganancia integral de dirección
<code>kd_steering</code>	0.10 rad·s/m	Ganancia derivativa de dirección
<code>max_steer_rad</code>	0.6 rad	Ángulo físico máximo del volante

4.4.4. Nodo generador de anotaciones (`annotation_generator_node`)

Este nodo actúa como capa de visualización del sistema. En cada *frame* de la cámara frontal, recoge el estado de todos los demás nodos y genera un mensaje `foxglove_msgs/ImageAnnotations` que Foxglove Studio superpone sobre la imagen en tiempo real. Las anotaciones incluyen:

- Polígono de carril: región semitransparente en verde que cubre el área entre las líneas izquierda y derecha del carril.
- Líneas de carril: línea izquierda en amarillo y derecha en cian, con grosor de 6 px.
- Vector de desviación: segmento del centro de la imagen al centro del carril estimado; verde si el error es menor de 30 px, rojo si es mayor.
- Bounding box de señal: rectángulo con esquinas decorativas y etiqueta de texto; naranja para límites de velocidad, rojo para señales de stop.
- HUD de telemetría: textos con el error lateral (m y px), la velocidad actual (km/h) y los valores de *throttle*, *brake* y *steer*.

La Figura 13 muestra la interfaz de Foxglove Studio.



Figura 13: Interfaz de Foxglove Studio durante una sesión de conducción autónoma.

4.5. Generación de datasets sintéticos

Una de las ventajas de trabajar con un simulador como CARLA es la posibilidad de generar datos etiquetados de forma automática. La cámara de segmentación semántica asigna a cada píxel una etiqueta de clase con precisión perfecta, ofreciendo un *ground truth* ideal para entrenar y validar los modelos de percepción sin ningún coste de anotación manual.

El paquete implementa dos nodos de recolección que operan en modo conducción manual (sin `vehicle_control_node`) a una cadencia de 5 Hz:

Dataset de carriles (`lane_dataset_node`). Detecta las marcas viales en la máscara semántica, las proyecta al espacio RGB y almacena en `dataset/lanes/` pares de imagen RGB y máscara binaria de *ground truth*.

Dataset de señales (`sign_dataset_node`). Localiza los píxeles de señales (color amarillo en CARLA), extrae bounding boxes y almacena en `dataset/signs/` las imágenes RGB junto con ficheros JSON de anotaciones.

Las composiciones Docker para la recolección están en `tools/docker-compose-lane-dataset.yaml` y `tools/docker-compose-sign-dataset.yaml`. El repositorio incluye también herramientas de visualización (`visualize_lanes_dataset.py`, `visualize_signs_dataset.py`) y scripts de anotación manual (`annotate_signs_dataset.py`, `label_signs_dataset.py`) para enriquecer el *dataset* sintético con etiquetas adicionales. Las animaciones del proceso de generación de datos (`Lanes_Dataset.gif` y `Signs_Dataset.gif`) están disponibles en el repositorio de GitHub.

4.6. Resultados

4.6.1. Rendimiento en tiempo real

La siguiente tabla recoge los tiempos de procesamiento medidos durante las sesiones de conducción autónoma en el simulador.

Cuadro 4: Latencia de inferencia medida durante la conducción autónoma.

Módulo	Latencia media	Frecuencia efectiva
Detección de carriles	≈ 50 ms	≈ 20 fps
Detección de señales	≈ 13 ms	≈ 77 fps

Ambas latencias se sitúan bien por debajo del período del controlador (100 ms a 10 Hz), lo que garantiza que los comandos de actuación siempre disponen de percepciones actualizadas.

4.6.2. Entorno de pruebas

Las pruebas se han realizado sobre el siguiente entorno hardware y software:

- CPU: Intel Core i7-13620H.
- GPU: NVIDIA GeForce RTX 5070 para portátiles.
- Sistema operativo: Ubuntu 22.04 (contenedor Docker).
- Framework: ROS2 Humble Hawksbill.
- Simulador: CARLA 0.9.15.

4.6.3. Conducción autónoma en simulación

El sistema completo mantiene el vehículo dentro del carril de forma continua a velocidades de hasta 30 km/h, adaptando la velocidad cuando detecta señales de límite escaladas a un tercio en el simulador (es decir, una señal de 90 km/h se interpreta como 30 km/h). El vídeo demostrativo del sistema funcionando en tiempo real (media/Demo.mp4) está disponible en el repositorio de GitHub:

<https://github.com/JoseLopez36/Navegacion-Deteccion-Senales>

5. Conclusiones

Este trabajo ha permitido diseñar e integrar un sistema de percepción completo para la conducción autónoma en simulación. En la detección de carriles, se comprobó que un enfoque basado en líneas horizontales no ofrecía la precisión necesaria para el seguimiento del vehículo, mientras que la segmentación semántica con VGG16–U-Net entrenada sobre TuSimple proporciona máscaras fiables en tramos rectos y con curvatura suave. En la detección de señales, la CNN clasifica correctamente los límites de 30 y 90 km/h (86 % y 81 % de acierto en el conjunto de prueba de la simulación), aunque

la clase de 60 km/h resulta más difícil (54 %) debido a la similitud visual entre los dígitos 6 y 9.

La integración de ambos módulos en una arquitectura ROS2 modular, desplegada en contenedores Docker sobre CARLA 0.9.15, demuestra que la combinación de modelos de *deep learning* con controladores PID es una solución eficaz y reproducible. El sistema opera en tiempo real con latencias medias de unos 50 ms en carriles y 13 ms en señales, y es capaz de mantener el vehículo dentro del carril adaptando su velocidad a los límites detectados. Como líneas de mejora futura, cabría reforzar la clasificación de la señal de 60 km/h, ampliar el conjunto de clases de señales y validar el sistema en escenarios más exigentes del simulador.